Processes



By:

Dr. H. K. Azad
Assistant Professor (Sr.)
SCOPE
Email: hiteshwarkumar.azad@vit.ac.in

Contents

- Process Concept
- Process Scheduling
- Operations on Processes
- Cooperating Processes
- Interprocess Communication
- Communication in Client-Server Systems

Process Concept

- An **operating system (OS)** executes many programs:
  - **Batch systems:** execute *jobs*
  - **Time-shared systems:** execute *user programs or tasks*
- The terms **job** and **process** are often used interchangeably.
- **Process**
  - A **program in execution**
  - Execution proceeds in a **sequential manner**
- **Program**
  - A **passive entity**: a set of instructions
  - Can range from a single line of code to millions of lines
- **A process consists of:**
  - **Program counter** – address of the next instruction
  - **Stack** – function calls, local variables
  - **Data section** – global and static variables

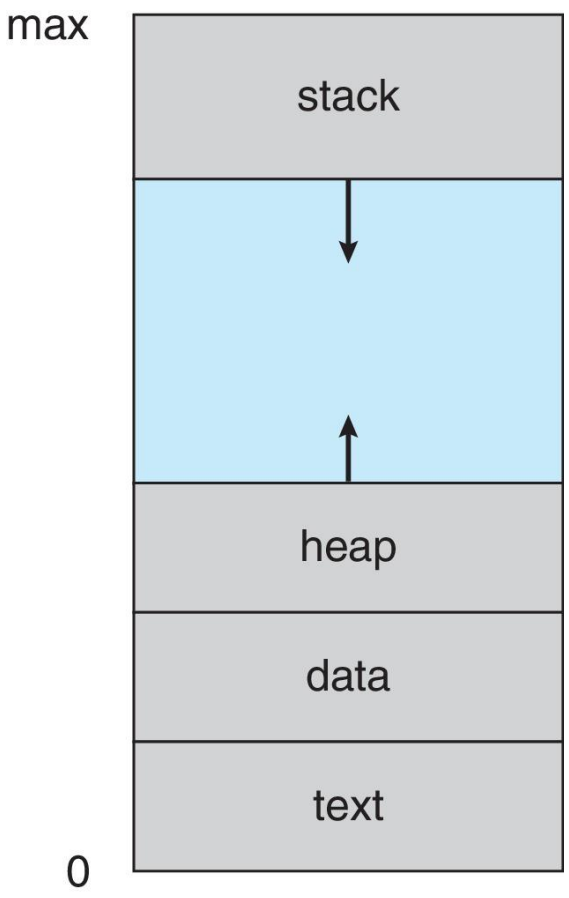
Process Concept (Cont.)

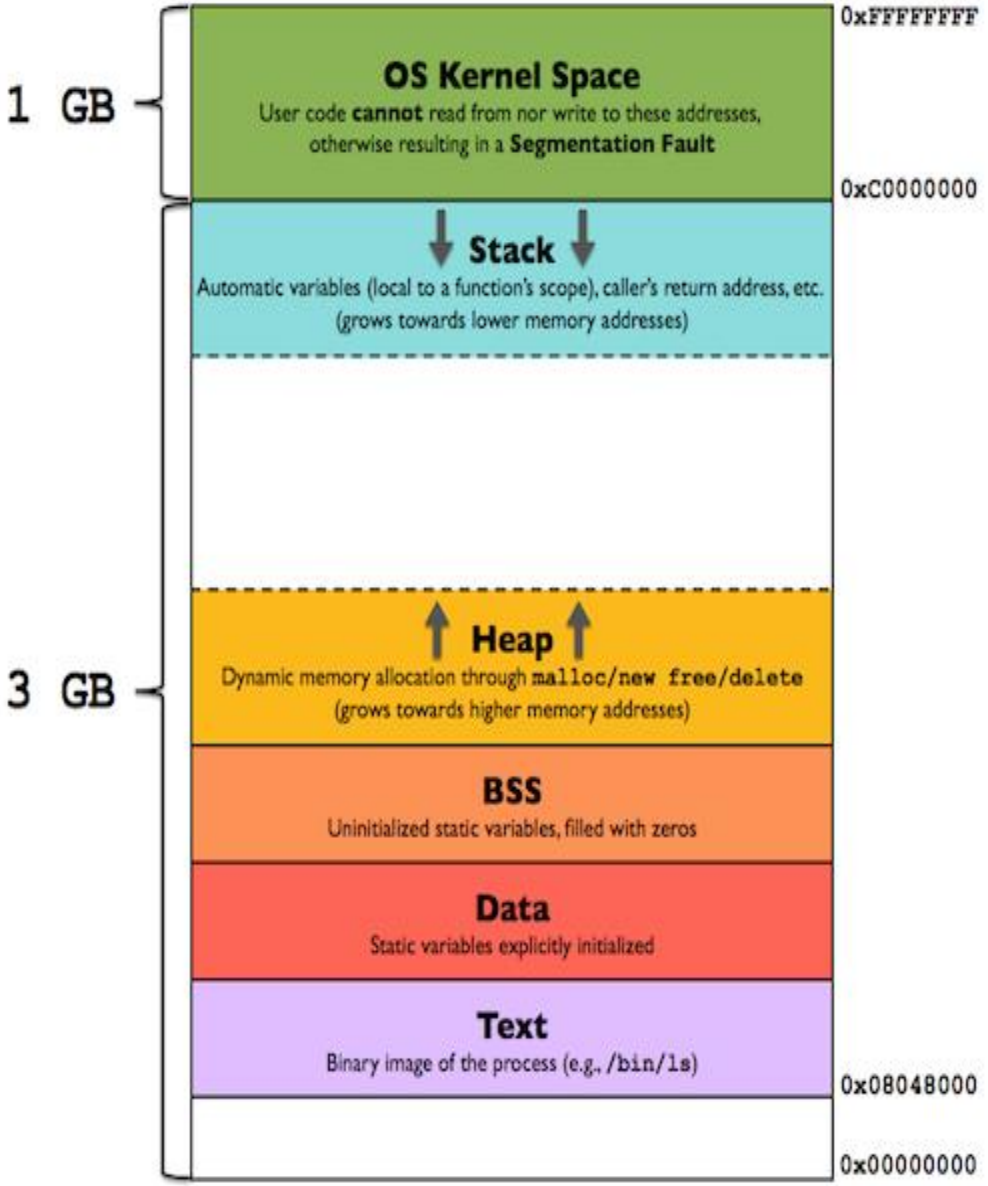
- An **operating system** executes multiple programs, each running as a **process**.
- **Process**
  - A **program in execution**
  - Execution proceeds **sequentially** within a single process
  - Instructions of a single process are **not executed in parallel**
- **Components of a Process**
  - **Text section**
    - Program code
  - **Program counter & CPU registers**
    - Current execution state
  - **Stack**
    - Temporary data
    - Function parameters, return addresses, local variables
  - **Data section**
    - Global and static variables
  - **Heap**
    - Dynamically allocated memory during runtime

Process Concept (Cont.)

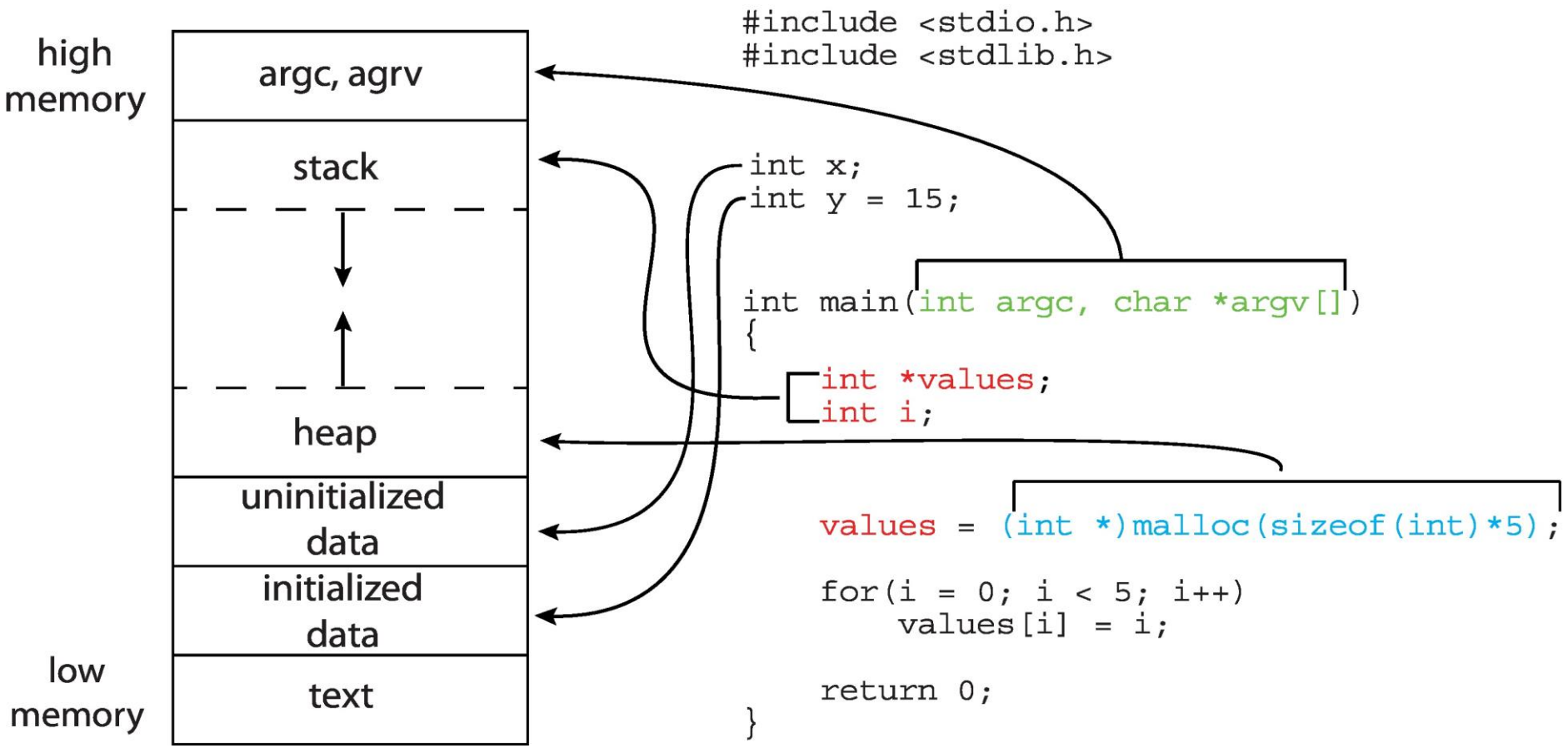
- **Program vs Process**
  - A **program** is a **passive entity** stored on disk (executable file)
  - A **process** is an **active entity** executing in memory
  - A program becomes a process when its executable is **loaded into memory**
- **Starting a Process**
  - Launched via:
    - GUI actions (mouse clicks)
    - Command-line execution
    - Scripts or system calls
- **Multiple Processes from One Program**
  - A single program can have **multiple processes**
  - Each process has its **own execution context**
  - Example:
    - Multiple users running the same:
      - **Compiler**
      - **Text editor**

Process in Memory





Memory Layout of a C Program

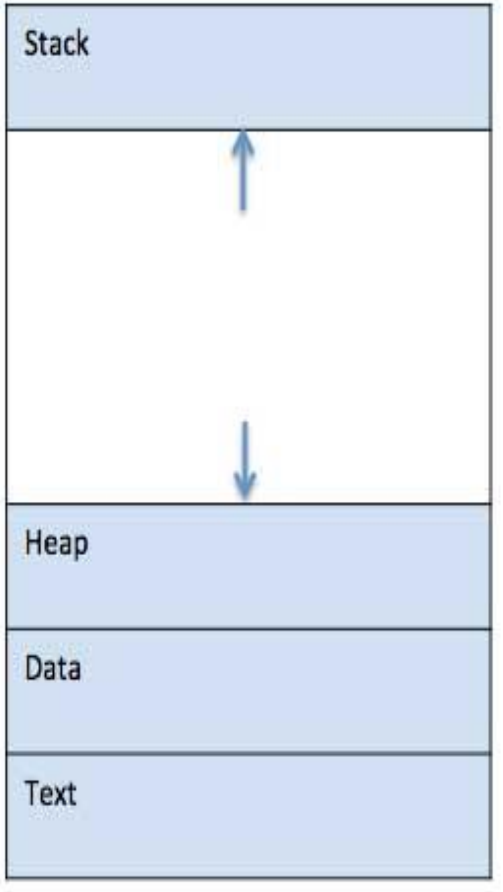


Layout of a Process in Main Memory

- When a program is loaded into memory and becomes a **process**, its address space is typically divided into **four sections**:
  - **Text**
  - **Data**
  - **Heap**
  - **Stack**

Memory Growth Direction:

- **Heap** grows **upward**
- **Stack** grows **downward**
- Stack and heap grow toward each other



Process Memory Components:

Component	Description
Text (Code)	Contains the executable program instructions.
Data	Stores global and static variables.
Heap	Memory allocated dynamically during runtime (e.g., `malloc`, `new`).
Stack	Stores temporary data such as function parameters, return addresses, and local variables.

Process State

- As a process executes, it **moves through different states** during its lifetime.

Common Process States:

- **New**
  - The process is being **created**
- **Ready**
  - The process is **waiting to be assigned to the CPU**
  - All required resources are available except the CPU
- **Running**
  - Instructions are **currently being executed** by the CPU
- **Waiting (Blocked)**
  - The process is **waiting for an event** to occur
  - Example: I/O completion, signal, resource availability
- **Terminated**
  - The process has **finished execution**
  - Resources are released

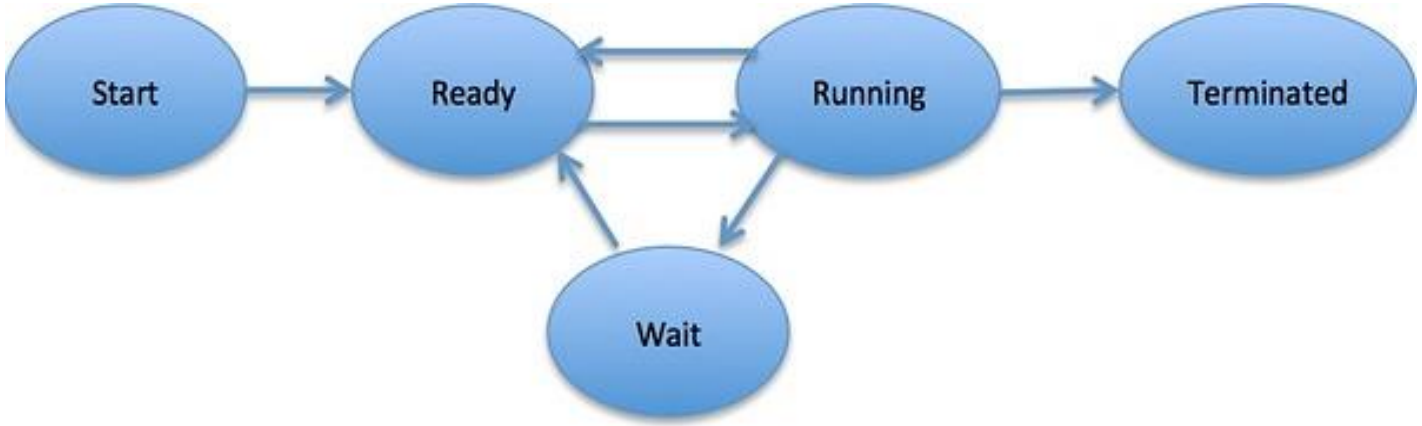
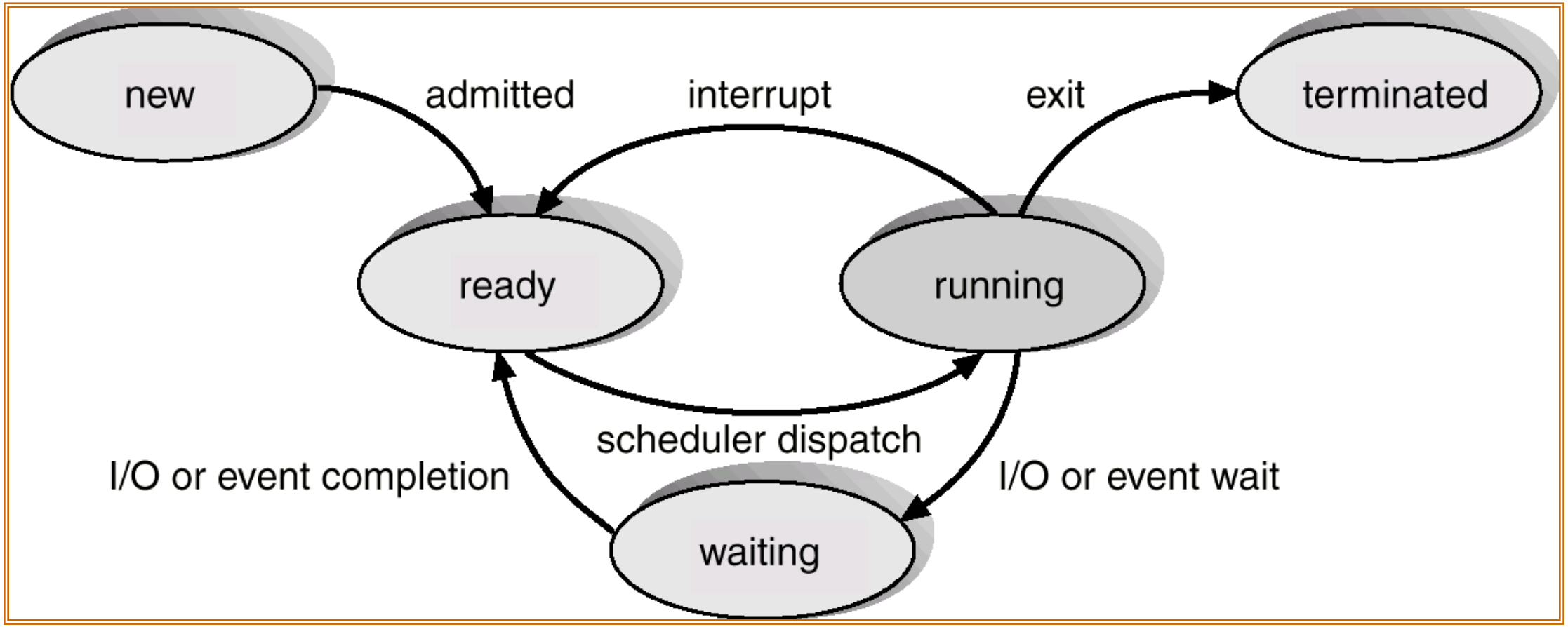


Diagram of Process State



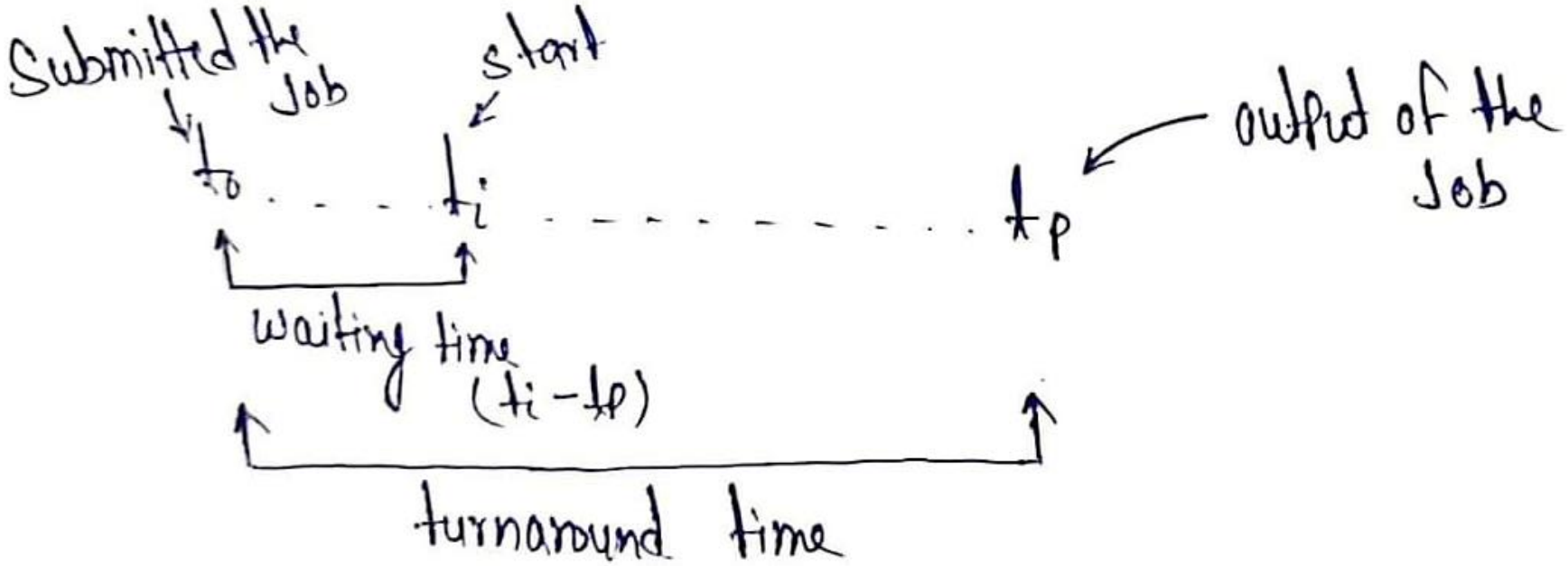
Process Timing Parameters

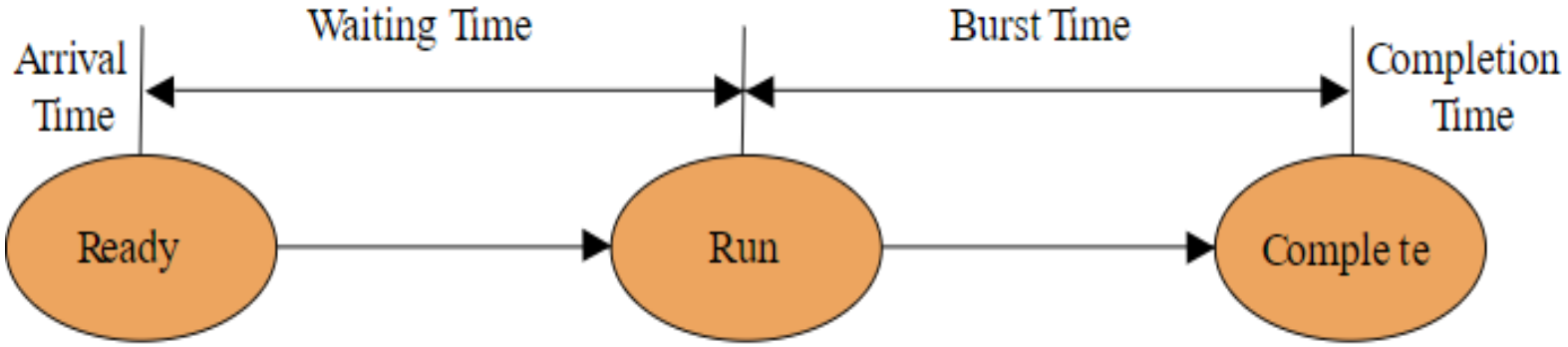
- **Arrival Time (AT)**
  - Time at which a process **enters the ready queue**
- **Burst Time (BT)**
  - Total **CPU execution time** required by the process
- **Completion Time (CT)**
  - Time at which the process **finishes execution**
- **Turnaround Time (TAT)**
  - Total time spent by the process in the system
  - **TAT = CT – AT**
- **Waiting Time (WT)**
  - Total time the process **waits in the ready queue**
  - **WT = TAT – BT**

Quick Tip

- **Turnaround Time = Waiting Time + Execution Time**
- These parameters are used to evaluate **CPU scheduling algorithms**

Waiting vs Turnaround time





TAT = CT - AT

CT - AT = WT + BT

Waiting Time = TAT - BT

Exponential Averaging

📌 What Is Exponential Averaging?

- Exponential averaging is a technique to predict the **next CPU burst time** based on past CPU bursts. It gives more weight to recent bursts while still considering historical data.

📊 Formula:

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$$

Where:

- τ_{n+1}: Predicted burst time for the next CPU burst
- t_n: Actual burst time of the most recent execution
- τ_n: Previously predicted burst time
- α: Weight factor ($0 \leq \alpha \leq 1$)

🧠 How It Works:

- If **α = 1**: prediction = most recent actual time (ignores history)
- If **α = 0**: prediction = previous prediction (ignores recent time)
- If **0 < α < 1**: balances recent time and history

📊 Example Calculation:

Let's assume:

- Initial estimate $\tau_0 = 10$
- $\alpha = 0.5$ (equal weight to recent and history)
- Actual burst times for 3 CPU bursts: 6, 8, 12

Step-by-step:

1. **First prediction:**

$$\tau_1 = 0.5 \cdot 6 + 0.5 \cdot 10 = 3 + 5 = 8$$

2. **Second prediction:**

$$\tau_2 = 0.5 \cdot 8 + 0.5 \cdot 8 = 4 + 4 = 8$$

3. **Third prediction:**

$$\tau_3 = 0.5 \cdot 12 + 0.5 \cdot 8 = 6 + 4 = 10$$

So, the OS estimates future CPU needs using this smoothed approach.

✅ **Advantages:**

- Simple and efficient
- Avoids abrupt changes in prediction
- Useful in dynamic environments (time-sharing OS)

⚠️ **Limitation:**

- Requires a good α value
- Not effective if process behaviour is highly unpredictable

FCFS vs SJF

Example:-

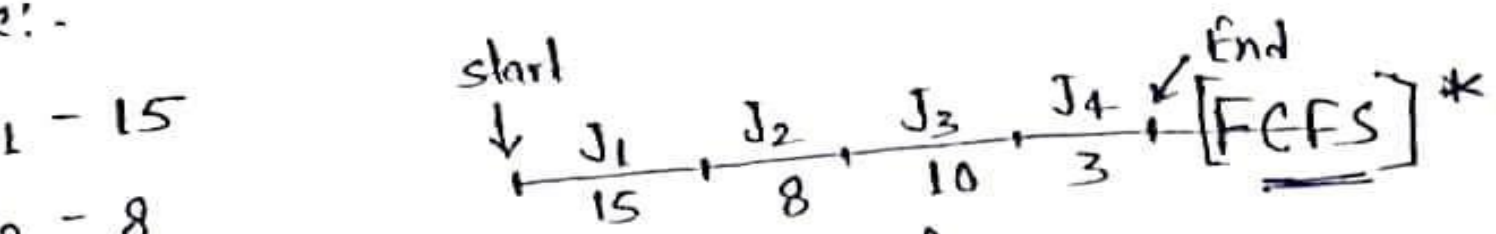
First- J_1 - 15

Second J_2 - 8

J_3 - 10

Last J_4 - 3

Job — Execution time



Waiting time of Job 1, $W_{J_1} = 0$

$W_{J_2} = 15$

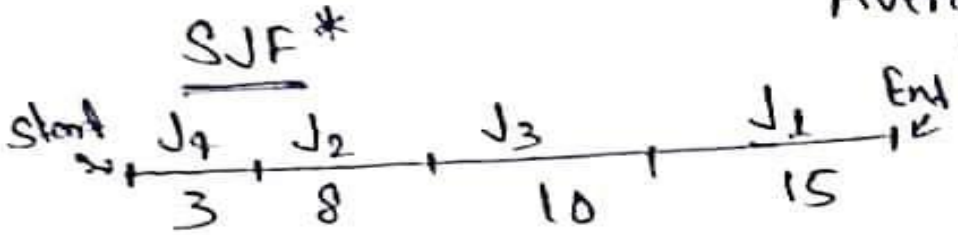
$W_{J_3} = 23$

$W_{J_4} = 33$

71

Average waiting time $t_{W_A} = \frac{71}{4}$

$= 17.75$



$W_{J_4} = 0$

$W_{J_2} = 3$

$W_{J_3} = 11$

$W_{J_1} = 21$

35

Average waiting time $t_{W_A} = \frac{35}{4}$

$= 8.75$

Consider the following table of arrival time and burst time for three processes P0, P1 and P2. (GATE-CS-2011)

Process	Arrival time	Burst Time
P0	0 ms	9 ms
P1	1 ms	4 ms
P2	2 ms	9 ms

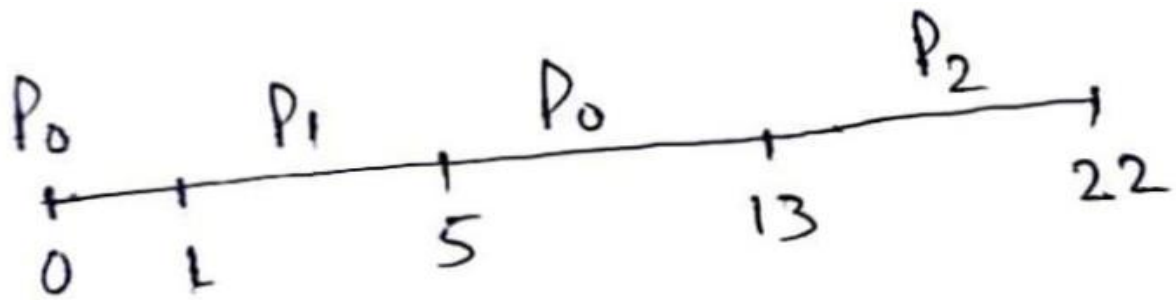
The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

(A) 5.0 ms

(B) 4.33 ms

(C) 6.33

(D) 7.33



$wP_0 = 0 + 4 = 4$

$wP_1 = 0$

$wP_2 = 13 - 2 = 11$

$tw_A = \frac{15}{3} = 5\,ms$

Consider the following set of processes, with the arrival times and the CPU-burst times given in milliseconds (GATE-CS-2004)

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	3
P4	4	1

What is the average turnaround time for these processes with the preemptive shortest remaining processing time first (SRPT) algorithm ?

(A) 5.50

(B) 5.75

(C) 6.00

(D) 6.25

An operating system uses the Shortest Remaining Time first (SRTF) process scheduling algorithm. Consider the arrival times and execution times for the following processes: (GATE-CS-2007)

Process	Execution time	Arrival time
P1	20	0
P2	25	15
P3	10	30
P4	15	45

What is the total waiting time for process P2?

(A) 5

(B) 15

(C) 40

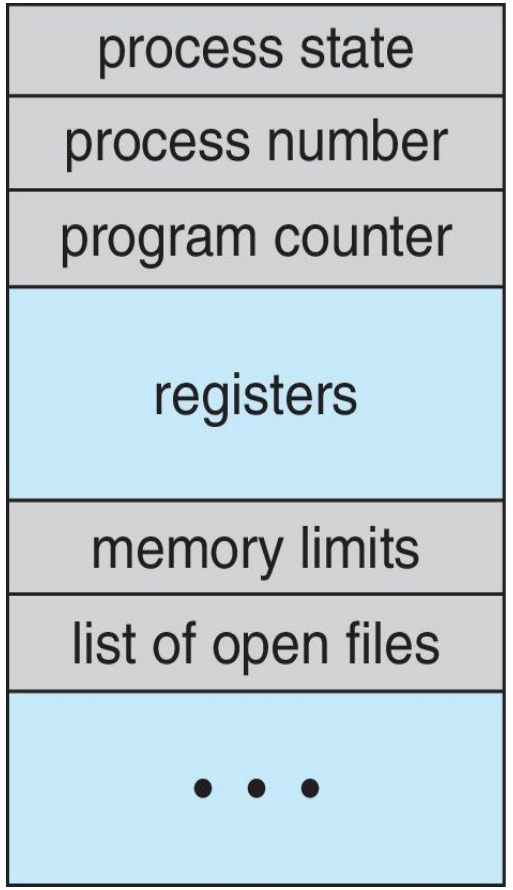
(D) 55

Process Control Block (PCB)

- The **Process Control Block** stores all information required by the OS to **manage and control a process**
- Also called **Task Control Block (TCB)** in some systems

Information Stored in a PCB:

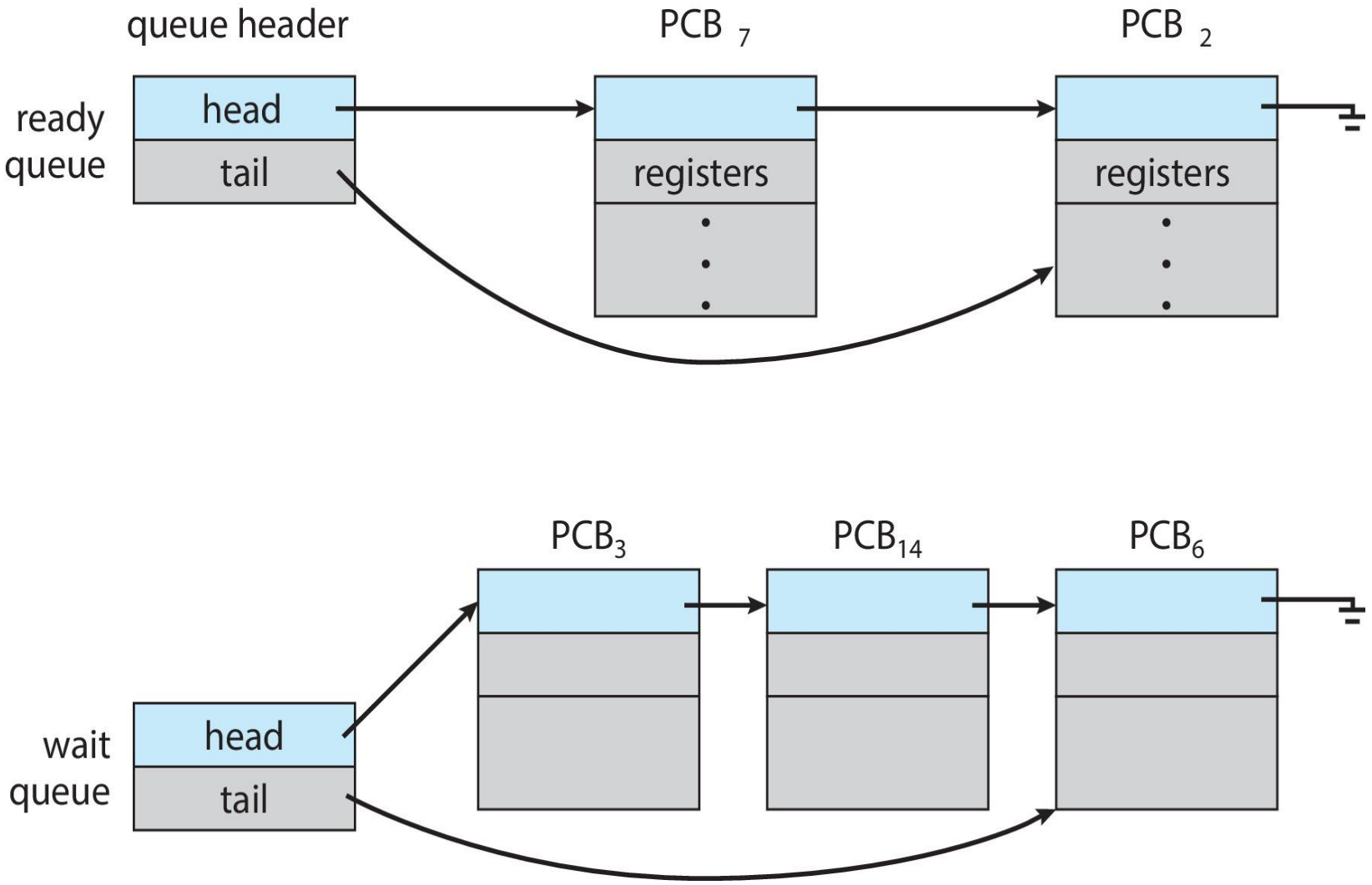
- **Process State**
  - Current state of the process (new, ready, running, waiting, terminated)
- **Process Identifier (PID)**
  - Unique number identifying the process
- **Program Counter**
  - Address of the **next instruction** to be executed
- **CPU Registers**
  - Contents of all **process-specific registers**
- **CPU Scheduling Information**
  - Priority, Scheduling queue pointers, Time slice (if applicable)
- **Memory Management Information**
  - Base and limit registers, page tables, or segment tables
- **Accounting Information**
  - CPU time used, Elapsed time, Time limits
- **I/O Status Information**
  - Allocated I/O devices, List of open files



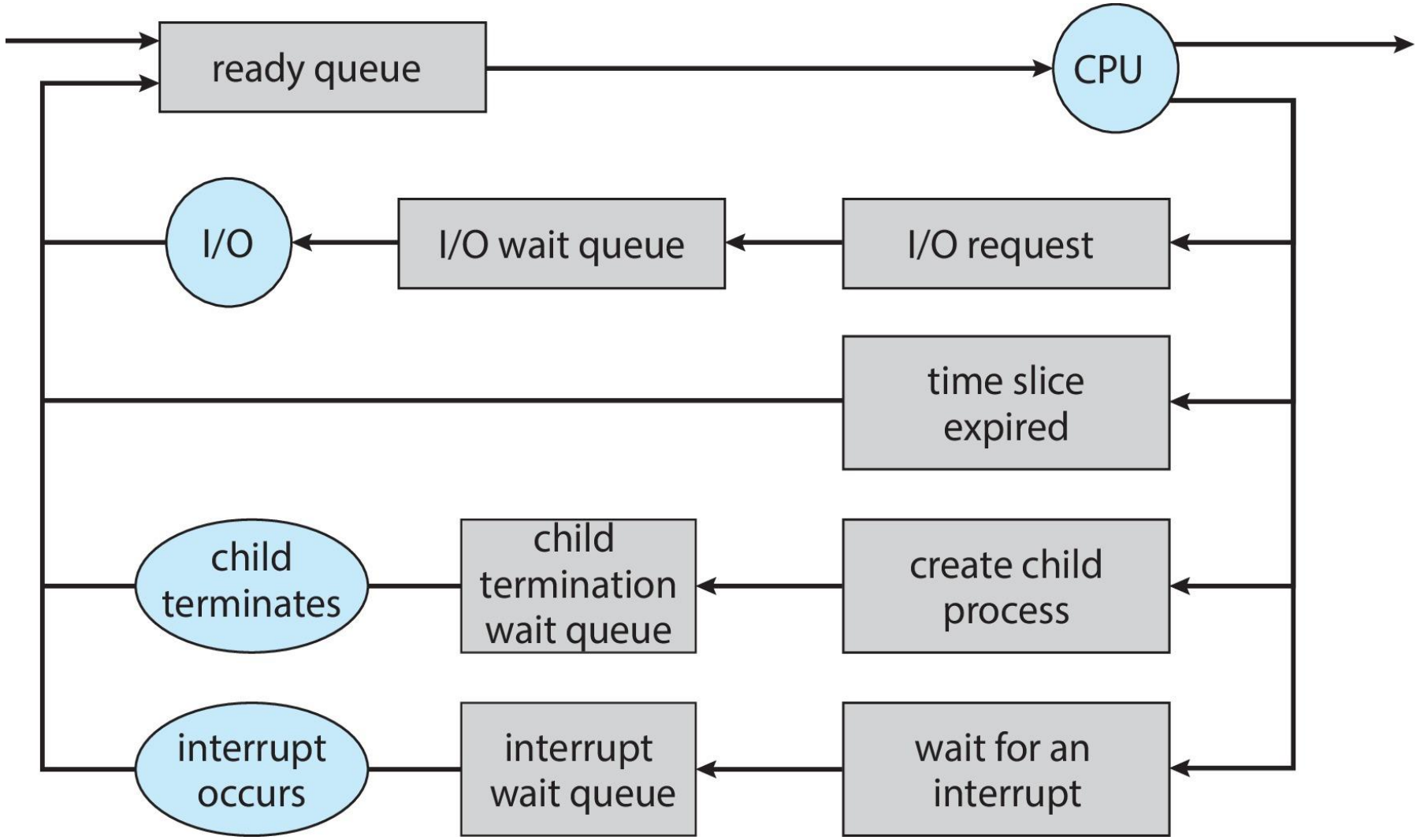
Process Scheduling

- The **process scheduler** selects one process from the **ready queue** to execute next on the **CPU core**.
- **Objectives of Scheduling**
  - Maximize **CPU utilization**
  - Improve **system throughput**
  - Minimize **waiting and response time**
  - Enable fast **context switching**
- **Scheduling Queues**
  - **Ready Queue**
    - Processes in main memory that are **ready to execute**
  - **Waiting (Blocked) Queues**
    - Processes waiting for an event (e.g., I/O completion)
- Processes **move between queues** as their state changes.

Ready and Wait Queues

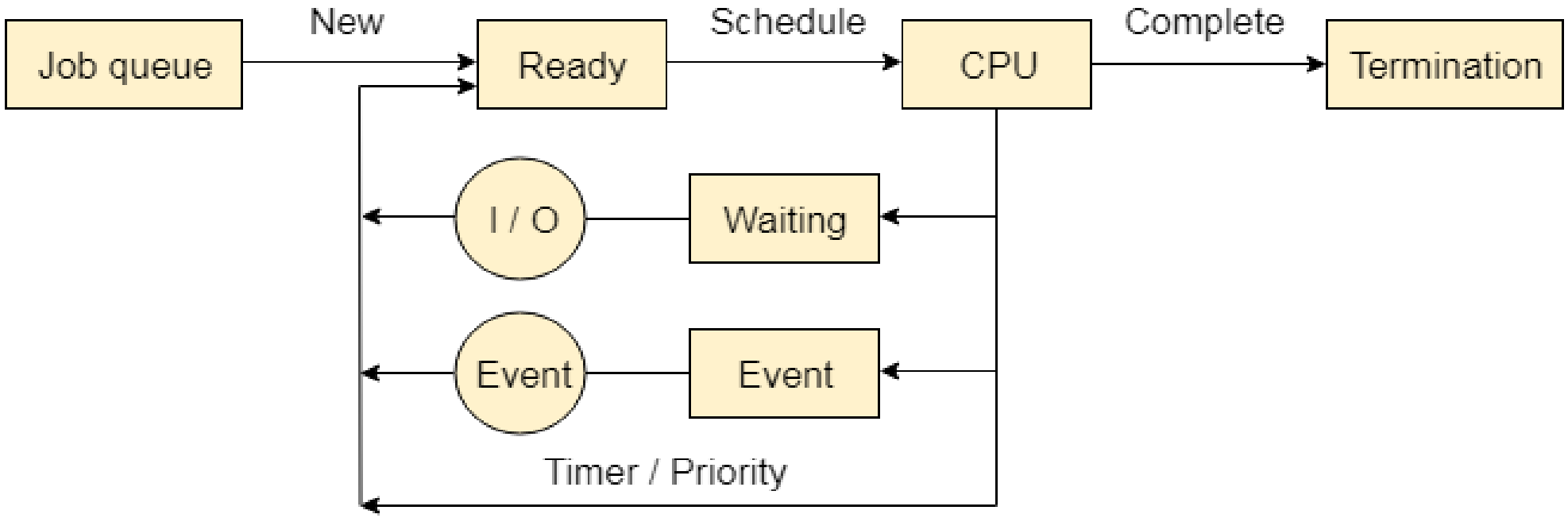


Representation of Process Scheduling

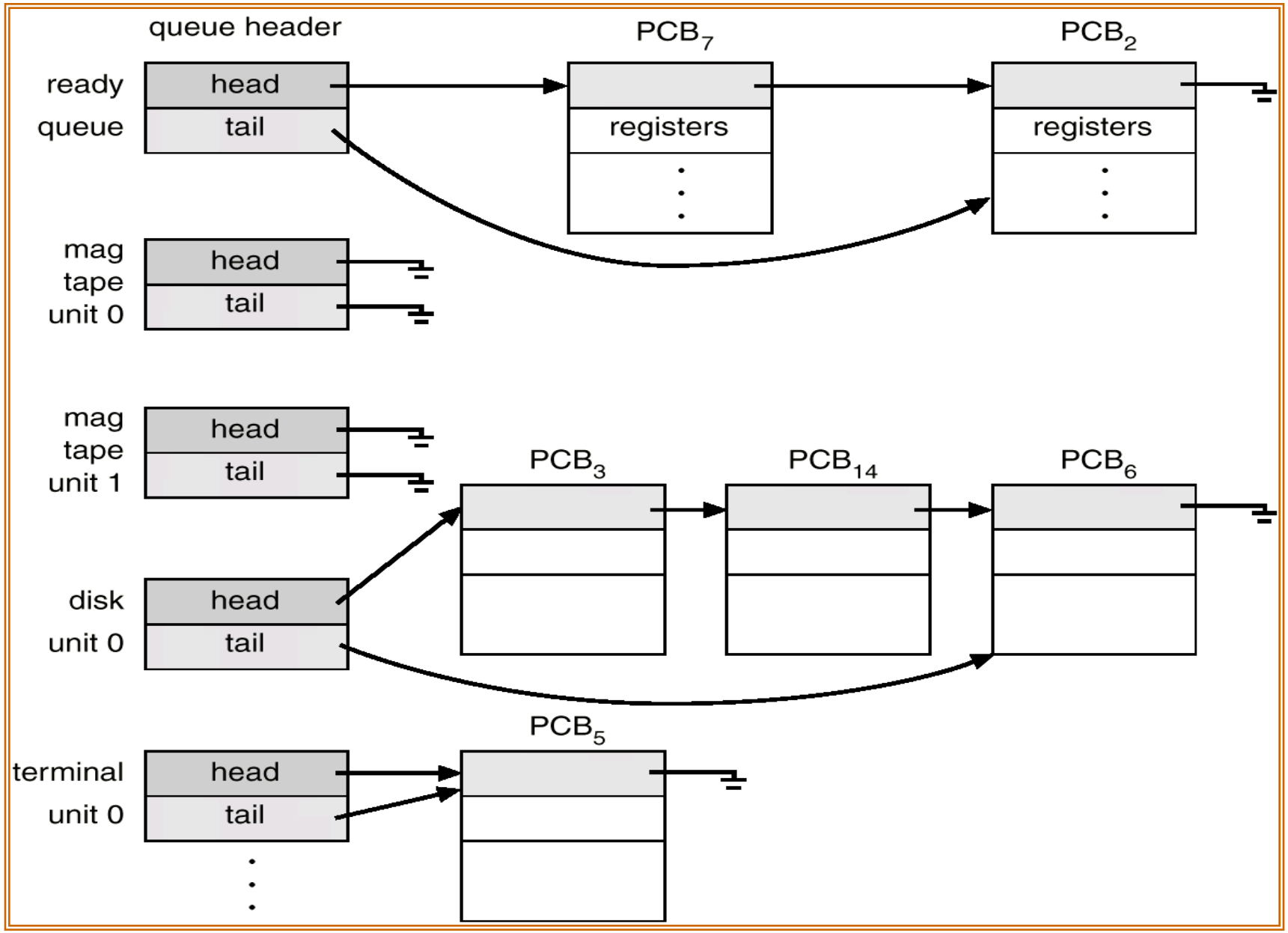


Process Scheduling Queues

- *Job queue* – set of all processes in the system
- *Ready queue* – set of all processes residing in main memory, ready and waiting to execute
- *Device queues* – set of processes waiting for an I/O device
- Process migration between the various queues



Ready Queue And Various I/O Device Queues



Context Switch

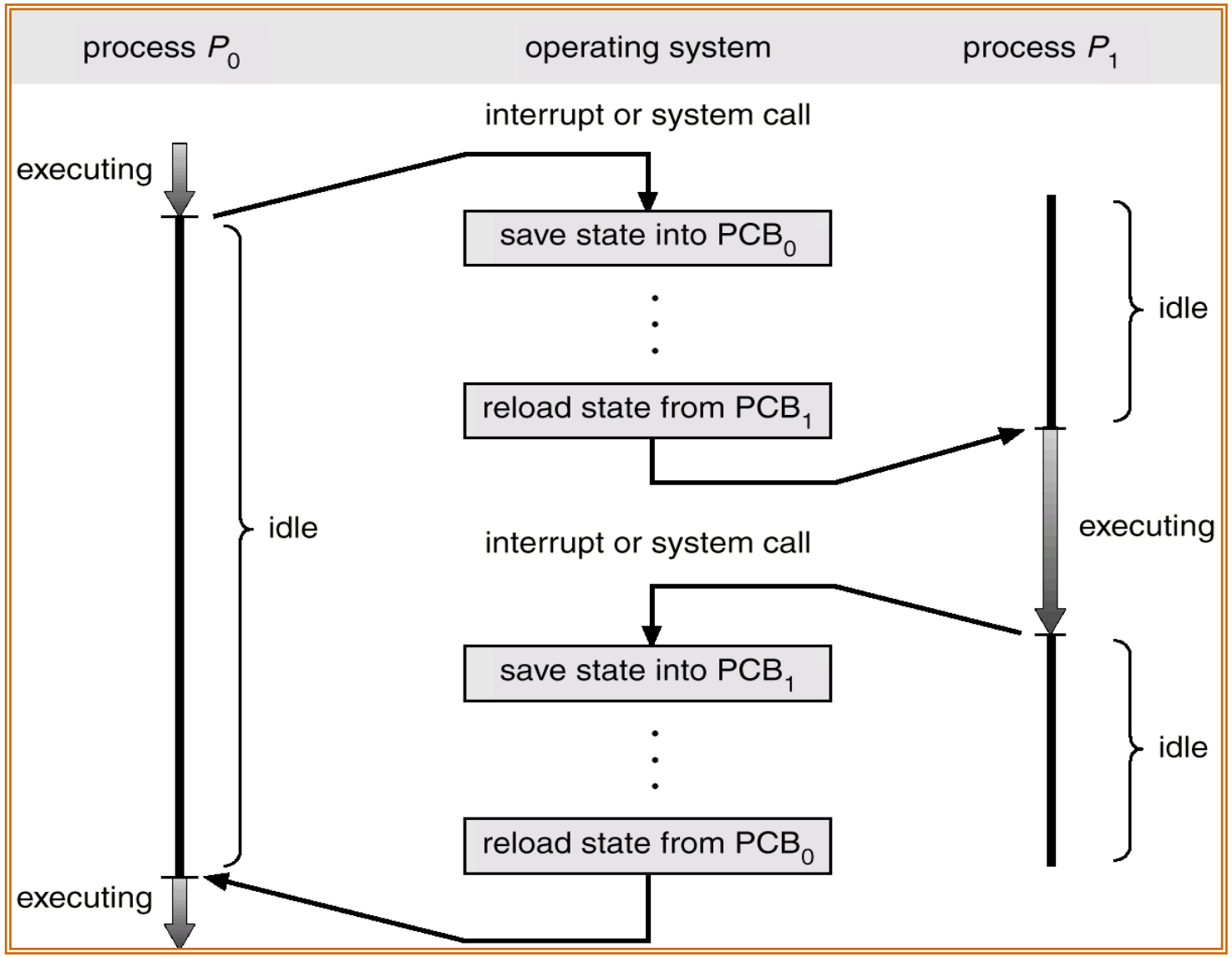
- A **context switch** occurs when the CPU switches from one process to another.
- The OS must:
  - **Save** the state of the currently running process
  - **Load** the saved state of the next process
- The **process context** (CPU state) is stored in the **Process Control Block (PCB)**.

Key Points

- **Context-switch time** is pure **overhead**
  - No useful computation is performed during the switch
  - More complex OS structures and larger PCBs ⇒ **longer context-switch time**
- **Hardware Support**
  - Context-switch time depends on hardware capabilities
  - Some CPUs provide **multiple sets of registers**, allowing faster context switches

CPU Switch From Process to Process

A context switch occurs when the CPU switches from one process to another.





START THE QUIZ

When a process moves from running to waiting state, this is usually due to:

A) Completion of the process

C) The arrival of a higher-priority process

B) A segmentation fault

D) An I/O request

Next

What part of the PCB is most directly involved in context switching?

A) Process priority

C) Memory limits

B) I/O queue pointers

D) CPU register values

Next

The Process Control Block (PCB) contains all of the following except:

A) Process ID

C) CPU Registers

B) Process State

D) Compiler Logs

Next

Which of the following is not a valid process state?

A) Compiling

C) Ready

B) Waiting

D) Executing

Next

Consider the following set of processes, with arrival times and the CPU burst times. If Shortest Remaining Time First (SRTF) scheduling is use, what will be the completion time of P1?

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

A) 16

C) 7

B) 17

D) 15

Next

What triggers a context switch?

A) System shutdown

C) Timer interrupt or I/O interrupt

B) Process completion

D) Cache miss

Well done!

Thanks